

LAPORAN PRAKTIKUM
POSTTEST 2
PEMROGRAMAN BERORIENTASI OBJEK



Disusun oleh:
Sabrina (2509106126)
Kelas (C2 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Penjelasan program

Program ini adalah simulasi sistem pendataan menu di restoran sushi. Program ini memungkinkan pengguna untuk: Mendata menu sushi (nama, harga, kategori, stok), Membuat pesanan pelanggan terhadap menu yang tersedia, Memproses pembayaran pesanan melalui kasir.

2. Struktur class

Program terdiri dari 3 class utama:

1. Class MenuSushi

```
2.     class MenuSushi:
3.         nama_restoran = "Sushi Ten"
4.         total_menu_terdaftar = 0
5.         pajak_persen = 10
6.
7.         __kode_internal_resto = "ST-INTERNAL-001"
8.
9.         def __init__(self, nama, harga, stok, kategori):
10.            self.nama = nama
11.            self.harga = harga
12.            self.kategori = kategori
13.            self._stok = stok
14.            self.__kode_menu = f"MENU-
{MenuSushi.total_menu_terdaftar + 1:03d}"
15.            MenuSushi.total_menu_terdaftar += 1
16.
17.            def tampilkan_info(self):
18.                print(f"[{self.kategori}] {self.nama} | Harga:
Rp{self.harga:,} | Stok: {self._stok}")
19.
20.            def kurangi_stok(self, jumlah):
21.                if jumlah <= 0:
22.                    print("Jumlah pengurangan tidak valid.")
23.                elif jumlah > self._stok:
24.                    print(f"Stok {self.nama} tidak cukup.")
25.                else:
26.                    self._stok -= jumlah
27.                    print(f"Stok {self.nama} berkurang {jumlah}. Sisa:
{self._stok}")
28.
29.            @property
30.            def stok(self):
31.                return self._stok
32.
```

```

33.         @stok.setter
34.         def stok(self, nilai_baru):
35.             if nilai_baru < 0:
36.                 raise ValueError("Stok tidak boleh negatif!")
37.             self._stok = nilai_baru
38.
39.         @classmethod
40.         def dari_dict(cls, data):
41.             return cls(data["nama"], data["harga"], data["stok"],
42. data["kategori"])
43.
44.         @classmethod
45.         def ubah_pajak(cls, pajak_baru):
46.             if pajak_baru < 0:
47.                 raise ValueError("Pajak tidak boleh negatif!")
48.             cls.pajak_persen = pajak_baru
49.
50.         @classmethod
51.         def cek_kode_internal(cls, kode):
52.             return kode == cls.__kode_internal_resto
53.
54.         @staticmethod
55.         def validasi_nama_menu(nama):
56.             return isinstance(nama, str) and len(nama.strip()) >= 3
57.
58.         @staticmethod
59.         def hitung_harga_setelah_pajak(harga):
60.             return int(harga * (1 + MenuSushi.pajak_persen / 100))
61.
62.     class SushiNigiri(MenuSushi):
63.         def __init__(self, nama, harga, stok, jenis_ikan,
64. is_premium=False):
65.             super().__init__(nama, harga, stok, "Nigiri")
66.             self.jenis_ikan = jenis_ikan
67.             self.is_premium = is_premium
68.
69.         def tampilkan_info(self):
70.             status = "PREMIUM" if self.is_premium else "Reguler"
71.             print(f"[{self.kategori}] {self.nama}
72. ({self.jenis_ikan}) | "
73. f"Harga: Rp{self.harga:,} | Stok: {self._stok} |
74. {status}")
75.
76.         def cek_kualitas(self):
77.             if self.is_premium and self.jenis_ikan.lower() in
78. ["salmon", "tuna", "uni"]:
79.                 print(f"{self.nama} kualitas ekspor - layak
80. disajikan.")
81.             else:
82.                 print(f"{self.nama} kualitas standar.")

```

```

77.
78.     class SushiRoll(MenuSushi):
79.         def __init__(self, nama, harga, stok, jumlah_potongan,
saus_khas):
80.             super().__init__(nama, harga, stok, "Roll")
81.             self.jumlah_potongan = jumlah_potongan
82.             self.saus_khas = saus_khas
83.
84.         def tampilkan_info(self):
85.             print(f"[{self.kategori}] {self.nama} |
{self.jumlah_potongan} potong | "
86.                 f"Saus: {self.saus_khas} | Harga: Rp{self.harga:,}
| Stok: {self._stok}")
87.
88.         def bagi_potongan(self, jumlah_orang):
89.             if jumlah_orang <= 0:
90.                 print("Jumlah orang tidak valid.")
91.             else:
92.                 per_orang = self.jumlah_potongan // jumlah_orang
93.                 print(f"{self.nama} dibagi {jumlah_orang} orang =
{per_orang} potong/orang.")
94.
95.     class Pesanan:
96.         nomor_pesanan_terakhir = 0
97.         status_default = "Menunggu"
98.
99.         def __init__(self, nama_pelanggan, menu, jumlah):
100.             Pesanan.nomor_pesanan_terakhir += 1
101.             self.id_pesanan = Pesanan.nomor_pesanan_terakhir
102.             self.nama_pelanggan = nama_pelanggan
103.             self.menu = menu
104.             self.jumlah = jumlah
105.             self.status = Pesanan.status_default
106.             self.__total_bayar = 0
107.
108.         def hitung_total(self):
109.             total = self.menu.harga * self.jumlah
110.             self.__total_bayar =
MenuSushi.hitung_harga_setelah_pajak(total)
111.             return self.__total_bayar
112.
113.         def proses_pesanan(self):
114.             if self.menu.stok >= self.jumlah:
115.                 self.menu.kurangi_stok(self.jumlah)
116.                 self.status = "Diproses"
117.                 print(f"Pesanan #{self.id_pesanan} sedang
diproses.")
118.             else:
119.                 print(f"Pesanan #{self.id_pesanan} gagal: stok tidak
cukup.")

```

```

120.
121.     def tampilkan_pesanan(self):
122.         print(f"Pesanan #{self.id_pesanan} |
{self.nama_pelanggan} | "
123.             f"{self.menu.nama} x{self.jumlah} | Status:
{self.status} | "
124.             f"Total: Rp{self.__total_bayar:,}")
125.
126.     @property
127.     def total_bayar(self):
128.         return self.__total_bayar
129.
130.     @total_bayar.setter
131.     def total_bayar(self, nilai):
132.         if nilai < 0:
133.             raise ValueError("Total bayar tidak boleh negatif!")
134.         self.__total_bayar = nilai
135.
136.     @classmethod
137.     def dari_dict(cls, data, daftar_menu):
138.         menu_ditemukan = None
139.         for m in daftar_menu:
140.             if m.nama.lower() == data["menu"].lower():
141.                 menu_ditemukan = m
142.                 break
143.         if menu_ditemukan is None:
144.             raise ValueError(f"Menu '{data['menu']}' tidak
ditemukan.")
145.         return cls(data["pelanggan"], menu_ditemukan,
data["jumlah"])
146.
147.     @classmethod
148.     def reset_nomor_pesanan(cls):
149.         cls.nomor_pesanan_terakhir = 0
150.
151.     @staticmethod
152.     def validasi_jumlah(jumlah):
153.         return isinstance(jumlah, int) and jumlah > 0

```

2. Class MenuSushi

```
3. class DaftarMenu:
4.     nama_instansi = "Sushi Ten Resto"
5.     total_daftar = 0
6.
7.     def __init__(self, nama_daftar):
8.         self.nama_daftar = nama_daftar
9.         self._koleksi_menu = []
10.        self.__total_harga_semua = 0
11.        DaftarMenu.total_daftar += 1
12.
13.    def tambah_menu(self, menu):
14.        if isinstance(menu, MenuSushi):
15.            self._koleksi_menu.append(menu)
16.            self.__total_harga_semua += menu.harga
17.            print(f"[+] {menu.nama} ditambahkan ke {self.nama_daftar}")
18.        else:
19.            print("Objek bukan MenuSushi, tidak bisa ditambahkan.")
20.
21.    def hapus_menu(self, nama_menu):
22.        awal = len(self._koleksi_menu)
23.        self._koleksi_menu = [
24.            m for m in self._koleksi_menu if m.nama.lower() !=
nama_menu.lower()
25.        ]
26.        if len(self._koleksi_menu) < awal:
27.            print(f"[-] {nama_menu} dihapus dari {self.nama_daftar}")
28.        else:
29.            print(f"Menu {nama_menu} tidak ditemukan.")
30.
31.    def tampilkan_semua(self):
32.        print(f"\n=== Daftar Menu: {self.nama_daftar} ===")
33.        for i, m in enumerate(self._koleksi_menu, 1):
34.            print(f"{i}. ", end="")
35.            m.tampilkan_info()
36.
37.    @property
38.    def total_harga_semua(self):
39.        return self.__total_harga_semua
40.
41.    @total_harga_semua.setter
42.    def total_harga_semua(self, nilai):
43.        if nilai < 0:
44.            raise ValueError("Total harga tidak boleh negatif!")
45.        self.__total_harga_semua = nilai
46.
47.    @classmethod
48.    def dari_dict(cls, data):
49.        return cls(data["nama_daftar"])
```

```

50.
51.     @staticmethod
52.     def validasi_nama_daftar(nama):
53.         return isinstance(nama, str) and len(nama.strip()) >= 3

```

3. Class MenuSushi

```

class Kasir:
    nama_instansi = "Sushi Ten Resto"
    total_transaksi = 0
    __pin_rahasia = "1234"

    def __init__(self, nama_kasir, shift):
        self.nama_kasir = nama_kasir
        self.shift = shift
        self.__saldo_kas = 0

    def terima_pembayaran(self, pesanan):
        total = pesanan.hitung_total()
        self.__saldo_kas += total
        Kasir.total_transaksi += 1
        pesanan.status = "Selesai"
        print(f"Kasir {self.nama_kasir} menerima Rp{total:,} dari pesanan  
#{pesanan.id_pesanan}")

    def tampilkan_saldo(self):
        print(f"Saldo kas {self.nama_kasir} (shift {self.shift}):  
Rp{self.__saldo_kas:,}")

    @property
    def saldo_kas(self):
        return self.__saldo_kas

    @saldo_kas.setter
    def saldo_kas(self, nilai):
        if nilai < 0:
            raise ValueError("Saldo kas tidak boleh negatif!")
        self.__saldo_kas = nilai

    @classmethod
    def dari_dict(cls, data):
        return cls(data["nama"], data["shift"])

    @classmethod
    def cek_pin(cls, pin):
        return pin == cls.__pin_rahasia

```

```

@staticmethod
def format_rupiah(angka):
    return f"Rp{angka:,}".replace(",", ".")

@staticmethod
def validasi_shift(shift):
    return shift.lower() in ["pagi", "siang", "malam"]

```

4. Class RiwayatTransaksi

```

class RiwayatTransaksi:

    def __init__(self, id_trx, nama_pelanggan, total):

        self.id_trx = id_trx

        self.nama_pelanggan = nama_pelanggan

        self.total = total

    def __str__(self):

        return f"[{self.id_trx}] {self.nama_pelanggan} - Rp{self.total:,}"

```

5. Class KasirDenganRiwayat

```

class KasirDenganRiwayat(Kasir):

    def __init__(self, nama_kasir, shift):

        super().__init__(nama_kasir, shift)

        self._riwayat = []

        self._counter_trx = 0

    def _buat_riwayat(self, nama_pelanggan, total):

        self._counter_trx += 1

        id_trx = f"TRX-{self._counter_trx:04d}"

        trx = RiwayatTransaksi(id_trx, nama_pelanggan, total)

```

```

self._riwayat.append(trx)

def terima_pembayaran(self, pesanan):
    super().terima_pembayaran(pesanan)
    self._buat_riwayat(pesanan.nama_pelanggan, pesanan.total_bayar)

def cetak_riwayat(self):
    print(f"\nRiwayat transaksi kasir {self.nama_kasir}:")
    for trx in self._riwayat:
        print(f" {trx}")

```

3. OUTPUT

```

SISTEM PENDATAAN MENU DI RESTORAN SUSHI
=====

--- 1. Membuat Objek MenuSushi & Subclass-nya ---
[Side Dish] Edamame | Harga: Rp15,000 | Stok: 30
[Nigiri] Salmon Nigiri (Salmon) | Harga: Rp35,000 | Stok: 20 | PREMIUM
[Nigiri] Tuna Nigiri (Tuna) | Harga: Rp30,000 | Stok: 15 | Reguler
[Roll] Dragon Roll | 8 potong | Saus: Spicy Mayo | Harga: Rp55,000 | Stok: 10
[Roll] California Roll | 6 potong | Saus: Wasabi | Harga: Rp45,000 | Stok: 12

```

```

--- 2. Method Unik Subclass ---
Salmon Nigiri kualitas ekspor - layak disajikan.
Tuna Nigiri kualitas standar.
Dragon Roll dibagi 4 orang = 2 potong/orang.
California Roll dibagi 3 orang = 2 potong/orang.

```

```

--- 3. Class Method & Static Method ---
Total menu terdaftar: 5
Pajak awal: 10%
Pajak setelah diubah: 15%
Cek kode internal 'ST-INTERNAL-001': True
Cek kode internal 'salah': False
Validasi nama 'Salmon': True
Validasi nama 'ab': False

```

```

--- 4. Agregasi: DaftarMenu ---
[+] Edamame ditambahkan ke Menu Utama
[+] Salmon Nigiri ditambahkan ke Menu Utama
[+] Tuna Nigiri ditambahkan ke Menu Utama
[+] Dragon Roll ditambahkan ke Menu Utama
[+] California Roll ditambahkan ke Menu Utama

```

```

=== Daftar Menu: Menu Utama ===
1. [Side Dish] Edamame | Harga: Rp15,000 | Stok: 30
2. [Nigiri] Salmon Nigiri (Salmon) | Harga: Rp35,000 | Stok: 20 | PREMIUM
3. [Nigiri] Tuna Nigiri (Tuna) | Harga: Rp30,000 | Stok: 15 | Reguler
4. [Roll] Dragon Roll | 8 potong | Saus: Spicy Mayo | Harga: Rp55,000 | Stok: 10
5. [Roll] California Roll | 6 potong | Saus: Wasabi | Harga: Rp45,000 | Stok: 12
Total harga semua menu: Rp180,000
[-] Edamame dihapus dari Menu Utama

=== Daftar Menu: Menu Utama ===
1. [Nigiri] Salmon Nigiri (Salmon) | Harga: Rp35,000 | Stok: 20 | PREMIUM
2. [Nigiri] Tuna Nigiri (Tuna) | Harga: Rp30,000 | Stok: 15 | Reguler
3. [Roll] Dragon Roll | 8 potong | Saus: Spicy Mayo | Harga: Rp55,000 | Stok: 10
4. [Roll] California Roll | 6 potong | Saus: Wasabi | Harga: Rp45,000 | Stok: 12

```

```

--- 5. Asosiasi: Pesanan ---
Stok Salmon Nigiri berkurang 2. Sisa: 18
Pesanan #1 sedang diproses.
Stok Dragon Roll berkurang 1. Sisa: 9
Pesanan #2 sedang diproses.
Pesanan #1 | Andi | Salmon Nigiri x2 | Status: Diproses | Total: Rp80,500
Pesanan #2 | Budi | Dragon Roll x1 | Status: Diproses | Total: Rp63,249

```

```

--- 6. Komposisi: Kasir & RiwayatTransaksi ---
Kasir Siti menerima Rp80,500 dari pesanan #1
Kasir Siti menerima Rp63,249 dari pesanan #2
Saldo kas Siti (shift Pagi): Rp143,749

Riwayat transaksi kasir Siti:
  [TRX-0001] Andi - Rp80,500
  [TRX-0002] Budi - Rp63,249

```

```

--- 7. Uji Getter & Setter ---
Stok awal nigiri1: 18
Stok setelah set valid (30): 30
Error set stok -5: Stok tidak boleh negatif!
Saldo kas awal: 143749
Saldo setelah set valid: 500000
Error set saldo -50000: Saldo kas tidak boleh negatif!

```

```

--- 8. Cek Pewarisan ---
nigiri1 adalah MenuSushi? True
nigiri1 adalah SushiRoll? False
SushiNigiri subclass MenuSushi? True
SushiRoll subclass MenuSushi? True

=====
PROGRAM SELESAI
=====

```

4. GITHUB

```

PS C:\Users\LENOVO\OneDrive\Documents\praktikum PBO> git init
Reinitialized existing Git repository in C:/Users/LENOVO/OneDrive/Documents/praktikum PBO/.git/
PS C:\Users\LENOVO\OneDrive\Documents\praktikum PBO> git add .
PS C:\Users\LENOVO\OneDrive\Documents\praktikum PBO> git commit -m "Finish posttest 1pbo"
On branch main
nothing to commit, working tree clean
PS C:\Users\LENOVO\OneDrive\Documents\praktikum PBO> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.43 KiB | 1.21 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/rina0706/PRATIKUM-PBO.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```